

Tutorial 3:

Building a Neural Network for MNIST Handwritten Digit Classification

Objectives:

- Train and Evaluate Neural Network
- Preprocessing the Data
- Visualizing the Data
- Solving Overfitting/Underfitting

Step 1 – Importing Necessary Libraries

First, we need to import the required libraries. These include numpy for numerical operations, matplotlib for visualization, and several modules from TensorFlow/Keras for building and training the neural network.

```
# Import necessary libraries
import numpy as np # For numerical operations
import matplotlib.pyplot as plt # For plotting and visualization
from tensorflow.keras.datasets import mnist # To import the MNIST dataset
from tensorflow.keras.models import Sequential # To create a sequential neural network model
from tensorflow.keras.layers import Dense, Flatten # For dense (fully connected) and flatten layers
from tensorflow.keras.utils import to_categorical # For one-hot encoding of labels
from tensorflow.keras.optimizers import Adam # For the Adam optimizer
```

Step 2 – Loading and Preprocessing the MNIST Dataset

This includes three steps

▣ Loading the Dataset:

- Download and load the MNIST dataset into training and test sets.

▣ Normalization:

- Scale the pixel values of the images from 0-255 to 0-1 to ensure stable and efficient training.

▣ One-Hot Encoding:

- Convert the labels from integers to binary matrices, making them compatible with the loss function used in multi-class classification.

For example, the label 3 is converted to [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]

```
# Step 1: Load and preprocess the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data() # Load the dataset

# Normalize the pixel values to the range [0, 1]
X_train = X_train.astype('float32') / 255.0 # Scale training data
X_test = X_test.astype('float32') / 255.0 # Scale test data

# One-hot encode the labels (0-9) for categorical classification
y_train = to_categorical(y_train, 10) # Encode training labels
y_test = to_categorical(y_test, 10) # Encode test labels
```

Step 3 –Building the Neural Network Model

We will build a neural network using the Sequential API. The network consists of:

1. A Flatten layer to convert the 2D images into a 1D vector.
2. Two Dense layers with ReLU activation for hidden units.
3. A final Dense layer with softmax activation for output.

```
# Step 2: Build the neural network model
model = Sequential([
    Flatten(input_shape=(28, 28)), # Flatten 28x28 images to a 1D vector of 784 features
    Dense(128, activation='relu'), # Hidden layer with 128 neurons and ReLU activation
    Dense(64, activation='relu'), # Hidden layer with 64 neurons and ReLU activation
    Dense(10, activation='softmax') # Output layer with 10 neurons (one for each digit) and softmax activation
])

# Print the model summary to understand the architecture
model.summary()
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Explanation of Each Column:

1. Layer (type):

- This column shows the name and type of each layer in the model.
- **flatten (Flatten):** Flattens the input image from a 2D shape (28x28) to a 1D vector of 784 elements.
- **dense (Dense):** A fully connected layer, also known as a Dense layer, where each neuron is connected to all neurons in the previous layer.

2. Output Shape:

- This column displays the shape of the output produced by each layer.
- The shape is given as (None, units), where None represents the batch size (which can vary) and units is the number of neurons in that layer.
- For example, (None, 128) means that the layer outputs 128 neurons for each sample in the batch.

3. Param #:

- This column indicates the total number of trainable parameters in each layer.
- Parameters include weights and biases learned during training.

1.

- Parameters include weights and biases learned during training.

Layer-by-Layer Breakdown:

1. Flatten Layer (flatten):

- **Layer Type:** Flatten
- **Output Shape:** (None, 784)
- **Param #:** 0
- **Explanation:** This layer converts each 28x28 pixel image into a 1D vector with 784 elements ($28 * 28 = 784$). It does not have any learnable parameters, as it only reshapes the input.

2. First Dense Layer (dense):

- **Layer Type:** Dense
- **Output Shape:** (None, 128)
- **Param #:** 100,480

- **Explanation:** This layer has 128 neurons. Each of the 784 input features (from the flattened layer) is connected to each of the 128 neurons, resulting in $784 * 128 = 100,352$ weights. Additionally, there are 128 bias terms, one for each neuron. Therefore, the total number of parameters is $100,352 + 128 = 100,480$.

3. Second Dense Layer (dense_1):

- **Layer Type:** Dense
- **Output Shape:** (None, 64)
- **Param #:** 8,256
- **Explanation:** This layer has 64 neurons. Each of the 128 neurons in the previous layer is connected to each of the 64 neurons in this layer, resulting in $128 * 64 = 8,192$ weights. Additionally, there are 64 bias terms, one for each neuron. Therefore, the total number of parameters is $8,192 + 64 = 8,256$.

4. Output Dense Layer (dense_2):

- **Layer Type:** Dense
- **Output Shape:** (None, 10)
- **Param #:** 650
- **Explanation:** This is the output layer with 10 neurons, one for each digit (0-9). Each of the 64 neurons in the previous layer is connected to each of the 10 output neurons, resulting in $64 * 10 = 640$ weights. Additionally, there are 10 bias terms, one for each output neuron. Therefore, the total number of parameters is $640 + 10 = 650$.

Step 4— Compiling the Model

Next, we compile the model. We specify:

Optimizer: Adam, which adapts the learning rate during training.

Loss Function: Categorical cross-entropy, which is suitable for multi-class classification.

Metrics: Accuracy, to track how well the model is performing.

```
# Step 3: Compile the model
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])
```

✓ 0.0s

Step 5 : Training the Model:

We will now train the model using the training data. We set:

- **Epochs:** 10, meaning the model will go through the training data 10 times.
- **Batch Size:** 32, meaning the model will update its parameters after every 32 samples.
- **Validation Split:** 20% of the training data will be used for validation to monitor performance

```
# Step 4: Train the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)
```

Step 6: Evaluating the Model:

After training, we evaluate the model on the test data to see how well it generalizes to unseen data.

```
# Step 5: Evaluate the model
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_accuracy:.4f}") # Print the test accuracy
```

✓ 0.5s

Step 7: Visualizing Training and Validation Performance

We will plot the training and validation accuracy and loss over the epochs to see how the model performed during training.

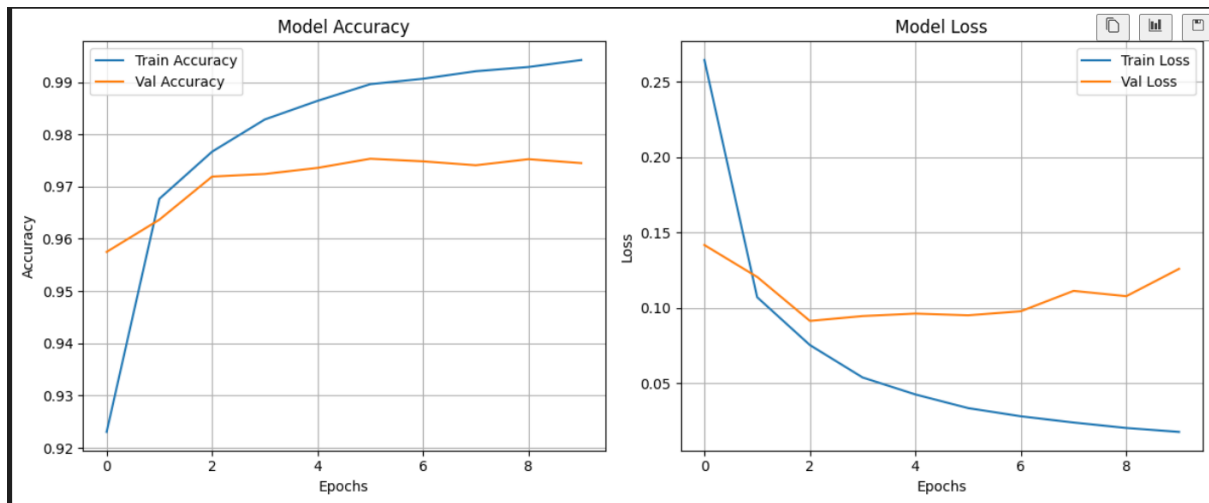
```
# Step 6: Visualize training and validation loss and accuracy
plt.figure(figsize=(12, 5)) # Create a figure with a specific size

# Plot training & validation accuracy values
plt.subplot(1, 2, 1) # Create a subplot for accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy') # Plot training accuracy
plt.plot(history.history['val_accuracy'], label='Val Accuracy') # Plot validation accuracy
plt.title('Model Accuracy') # Set the title
plt.xlabel('Epochs') # Set x-axis label
plt.ylabel('Accuracy') # Set y-axis label
plt.legend() # Show legend
plt.grid() # Add grid lines

# Plot training & validation loss values
plt.subplot(1, 2, 2) # Create a subplot for loss
plt.plot(history.history['loss'], label='Train Loss') # Plot training loss
plt.plot(history.history['val_loss'], label='Val Loss') # Plot validation loss
plt.title('Model Loss') # Set the title
plt.xlabel('Epochs') # Set x-axis label
plt.ylabel('Loss') # Set y-axis label
plt.legend() # Show legend
plt.grid() # Add grid lines

plt.tight_layout() # Adjust subplots to fit into figure area.
plt.show() # Display the plots
```

Output:



Step 8: Making Predictions and Visualizing Results

We use the trained model to make predictions on the test set. We then visualize some test images along with their true and predicted labels.

```

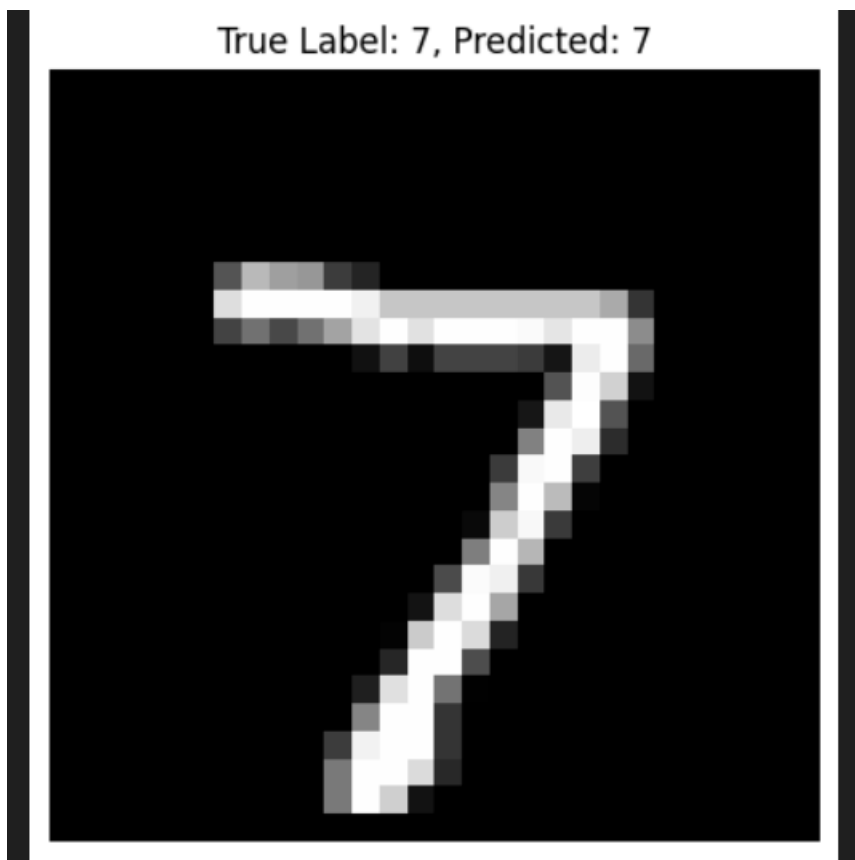
# Step 7: Make predictions on the test set
predictions = model.predict(X_test)

# Display the first test image and its predicted label
plt.figure(figsize=(5, 5)) # Create a figure for the image
plt.imshow(X_test[0], cmap='gray') # Show the first test image in grayscale
plt.title(f"True Label: {np.argmax(y_test[0])}, Predicted: {np.argmax(predictions[0])}") # Display true and predicted labels
plt.axis('off') # Hide the axis
plt.show() # Display the image

# Display a grid of images with true and predicted labels
num_images = 9 # Number of images to display
plt.figure(figsize=(10, 10)) # Create a figure for the grid of images
for i in range(num_images):
    plt.subplot(3, 3, i + 1) # Create a 3x3 grid of subplots
    plt.imshow(X_test[i], cmap='gray') # Show each test image in grayscale
    plt.title(f"True: {np.argmax(y_test[i])}, Predicted: {np.argmax(predictions[i])}") # Display true and predicted labels
    plt.axis('off') # Hide the axis
plt.tight_layout() # Adjust subplots to fit into figure area.
plt.show() # Display the grid of images

```

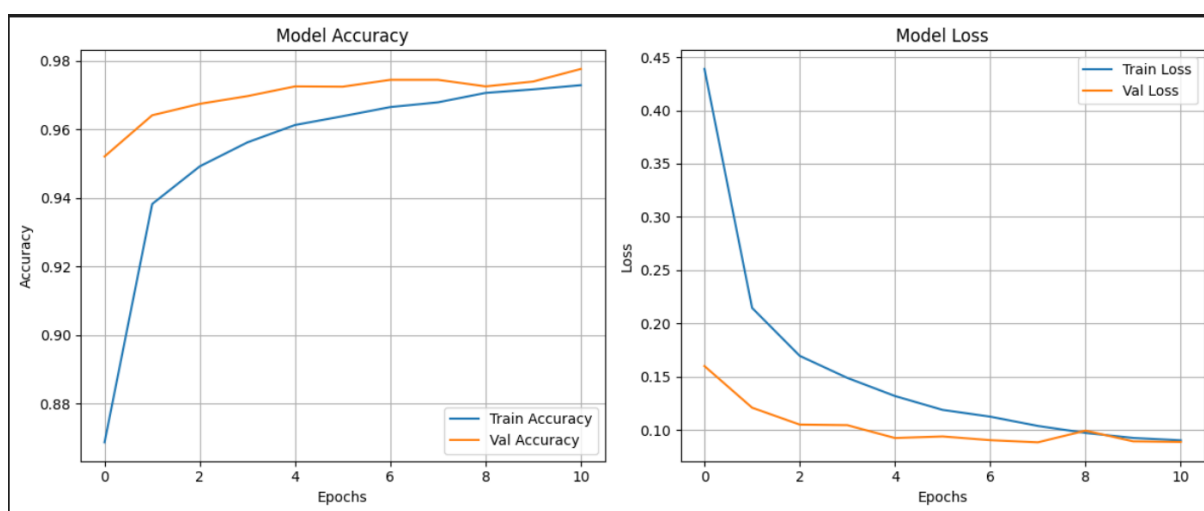
Output



Tasks:

1. Experiment with Different Architectures
 - Modify the neural network architecture to see how it affects model performance.
 - Instructions: Add more layers or change the number of neurons in each layer.
 - Experiment with different activation functions such as tanh or sigmoid.
 - Compare the performance of the modified model with the original model.
2. Compare Different Optimizers
 - Test different optimizers to see how they impact training and convergence.
 - Train the model with different optimizers such as SGD, RMSprop, and Adam.
 - Compare training speed and final accuracy of each optimizer.
3. Observe and interpret the training and validation accuracy and loss curves.
 - Identify whether the model is *overfitting, *underfitting, or well-fitted by comparing the curves.
 - Observe the effect of changing the number of epochs on overfitting and underfitting.
 - Implement Early Stopping
 - Experiment with Regularization Techniques

Your model after improvement should look similar or close to the following graphs.



***Overfitting:** Training accuracy is much higher than validation accuracy, and the validation loss increases or stagnates after some epochs.

***Underfitting:** Both training and validation accuracy are low, and training loss is high.

Additional Notes:

Understanding Cross Entropy Loss:

Cross-Entropy Loss is a commonly used loss function for classification tasks, particularly in multi-class and binary classification problems. It measures the difference between the true label distribution and the predicted probability distribution output by the model.

1. Binary Cross-Entropy Loss (for binary classification):

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where:

- N is the number of samples.
- y_i is the true label (0 or 1).
- p_i is the predicted probability for class 1.

2. Categorical Cross-Entropy Loss (for multi-class classification):

$$L = - \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(p_{ij})$$

where:

- N is the number of samples.
- C is the number of classes.
- y_{ij} is 1 if sample i belongs to class j , otherwise 0.
- p_{ij} is the predicted probability that sample i belongs to class j .

Example: Multi-Class Classification with 3 Classes

Suppose we have a classification problem with 3 classes: A, B, and C. For a single sample, the true label and the model's predicted probabilities are as follows:

- True Label (One-Hot Encoded):

$$y = [1, 0, 0]$$

This means the true class is "A".

- Predicted Probabilities:

$$p = [0.7, 0.2, 0.1]$$

This indicates the model predicts a 70% probability for class A, 20% for class B, and 10% for class C.

Calculating the Loss:

1. For Class A:

$$y_1 = 1, \quad p_1 = 0.7$$

Contribution to the loss:

$$-(1) \log(0.7) = -\log(0.7) = 0.357$$

2. For Class B:

$$y_2 = 0, \quad p_2 = 0.2$$

Contribution to the loss:

$$-(0) \log(0.2) = 0$$

3. For Class C:

$$y_3 = 0, \quad p_3 = 0.1$$

Contribution to the loss:

$$-(0) \log(0.1) = 0$$

Total Loss for this Sample:

$$L = 0.357 + 0 + 0 = 0.357$$

Interpretation:

The loss value of 0.357 indicates how well the model's predicted probabilities match the true class. A lower loss indicates a better match. If the predicted probability for the true class (A) were lower, the loss would be higher, penalizing the model more.